

Solution - Task 2 - QA Strategy & Thinking:

1. Where do you start?

- Firstly I would start exploring the app as a user without any deep technical knowledge. This helps to understand how smooth the user experience is and catch the bugs.
- Secondly, Go through product PRD's (maybe use AI tools to get summary), meet Product Managers and Dev to understand the core user workflows and business rules.
- Identifying critical workflows involving user funds, wallets/FX - like balances, transactions, failed payments
- Identify
- Understanding the system architecture, APIs, environments, and any third-party integrations and their dependencies.
- Reviewing existing requirements, user stories, designs, and known issues.
- Setting up the testing environment and ensuring stable test accounts are available.
- Creating a risk-based testing strategy, prioritizing features that could result in financial loss or security issues or maintaining Company reputation and customer trust

Since release is two weeks and has no existing QA process, I would focus on building confidence in the highest-risk functionality instead of trying to achieve complete test coverage based on prioritization.

2. How would you approach testing this app?

Functional Testing: Verify core user features, such as authentication, account management, trading, wallet operations, portfolio, and notifications, work as expected.

Risk-Based Testing: Prioritize testing critical financial workflows, including order execution, balance calculations, deposits, withdrawals, and payment failures, to prevent issues affecting user funds.

API Testing: Validate APIs for authentication, trading, portfolio, market data, error handling, and rate limiting to ensure backend reliability and faster feedback.

Mobile Testing: Test the app across iOS and Android devices, different screen sizes, orientations, network conditions, and app lifecycle scenarios to ensure a consistent user experience.

Security Testing: Verify authentication, authorization, session management, secure data handling, API security, and input validation to protect user accounts and financial information.

Performance Testing: Assess application responsiveness, API performance, trading latency, market data updates, and handling of large portfolios under expected load.

3. What does QA look like inside a sprint, from ticket creation through to regression?

User story Refinement: Review requirements, clarify acceptance criteria, identify edge cases, and estimate testing effort.

Development: Developers build the feature while QA prepares test cases, test data, and automation scenarios. (automate test cases if possible)

Feature Testing: Validate new functionality through functional, negative, UI, and API testing. (Run automation if test cases are automated), raise bugs if any and verify again once fixed.

Automation: Automate smoke tests, critical user journeys, and regression scenarios to provide fast feedback.

Regression: Execute smoke and regression tests, verify bug fixes, and review any test failures before release -> verify if any impact to other/shared services (in case we focus on a particular service). Based on test results - all critical tests pass, no blocker defects remain, and the release readiness checklist is complete , provide Go or no go for the release

Release: After the release to production immediately perform a quick test on PROD if possible Or monitor the prod logs, to identify issues early and avoid any big impact to customers.

Sprint retrospective: Have a sprint retro to discuss what went well and what not during the sprint, apply these feedback loops to help improve sprints.

4. What does your ideal regression suite look like?

The regression suite focuses on the application's most critical business workflows to ensure stability after every release. It includes authentication, trading operations, wallet transactions, portfolio management, navigation, and key API validations. Tests are executed on both **iOS and Android**, with priority given to critical user journeys such as login, order execution, deposits/withdrawals, and balance updates.

Regression testing is an ongoing process. After each release, tests covering newly implemented features should be added to the regression suite, allowing future releases to validate both existing and newly delivered functionality.

5. What would keep you up at night about this app specifically and releasing to the public?

Incorrect Balance: Account balances and portfolio values do not accurately reflect completed transactions.

Duplicate transactions: Network retries or system issues cause the same trade or payment to be processed multiple times.

Security vulnerabilities: Unauthorized access or exposure of sensitive user and financial data.

App stability: Crashes or freezes during critical actions, such as placing or managing trades.
Company reputation: Critical production issues can lead to financial loss, loss of customer trust, negative publicity, and damage to the company's reputation.

Test Plan:

Feature	Priority	Test
Login	High	Functional, Security
Wallet	Critical	Functional, API
Trading	Critical	Functional, API
Portfolio	High	Functional, API
Notifications	Medium	Functional
Settings	Low	Functional
Security	Critical	Functional
Performance	High	Performance

A release readiness checklist:

1. All critical user stories completed.
2. No open Critical or High severity defects.
3. Sanity tests pass.
4. Regression suite passes.
5. API tests pass.
6. Trading workflows validated.
7. Wallet functionality verified.
8. Performance testing passed.
9. Monitoring and logging enabled.
10. Rollback plan documented.
11. Production deployment approved.

A risk matrix

Each feature is assigned a risk level based on two factors: how frequently it is used and the business impact of a failure. Features with high usage and high impact receive the highest testing priority. The below matrix can be updated accordingly.

Feature	Impact	Usage	Risk score
---------	--------	-------	------------